

AI Study Assistant

Project Charter & Technical Architecture

Table of Contents

01	Project Overview	3
02	System Architecture	4
03	Development Phases	6
04	Team Responsibilities	8
05	GitHub & Engineering Standards	9

01 / Project Overview

1.1 Executive Summary

AI Study Assistant is a learning platform that lets a student upload their own study material – PDFs, DOCX, PPTX, slides, scanned notes – and then converse with it, generate revision content from it, and plan study time around it. Instead of a generic chatbot, every answer is grounded in the student's own uploaded documents and cited back to the exact page it came from.

1.2 Problem Statement

- Study material is scattered across drives, WhatsApp, PDFs, and printed notes – nothing is searchable in one place.
- Revising before an exam means re-reading everything; there is no fast way to query "what does my notes say about X".
- Making quizzes, flashcards, and summaries by hand is slow and gets skipped under deadline pressure.
- General-purpose AI chatbots (ChatGPT etc.) don't persist a student's material, don't cite the source page, and hallucinate when the topic is niche or lecture-specific.
- There is no structured way to turn "10 subjects, 3 weeks left" into an actual day-by-day plan.

1.3 What the Product Does

Feature	Description
Material Upload	PDF, DOCX, PPTX, images (OCR), organized by subject
Chat with Notes	Ask questions, get answers grounded in the uploaded material with page-level citations
Summarizer	Short / detailed / chapter-wise summaries on demand
Quiz Generator	Auto-generated MCQ, fill-in-the-blank, true/false with scoring
Flashcard Generator	Question/answer pairs with difficulty tagging, for spaced revision
Study Planner	Day-by-day plan generated from subjects, deadlines, and available hours
Progress Tracking	Analytics on what's been studied, quiz scores over time, weak topics

1.4 Target Users

Primary: college students preparing for semester exams and placement interviews who already have a pile of unorganized notes and slides. Secondary: competitive-exam aspirants who need to convert long syllabi into structured revision material.

Core value proposition – your own notes, made searchable, quizzable, and plannable, with every answer traceable back to the exact source page instead of a generic AI guess.

02 / System Architecture

2.1 Why This Architecture

- **Three decoupled layers** (Frontend / Backend API / AI Service) map directly onto the three-person team, so each person owns a layer with a clear API contract instead of stepping on each other's code.
- **RAG instead of fine-tuning** – cheaper, avoids hallucination because answers are grounded in retrieved chunks with citations, and updates instantly the moment a new file is uploaded (no retraining cycle).
- **Separate vector database** – relational databases are not built for approximate nearest-neighbor search over embeddings; a dedicated vector store (ChromaDB/Qdrant) makes semantic search fast at scale.
- **Free/open-source-first stack** – FastAPI, PostgreSQL, ChromaDB, and Next.js are all free and self-hostable; the only real running cost is LLM API calls, which have a usable free tier (Gemini 2.5 Flash) – important for a student project with no funding.

2.2 High-Level Architecture

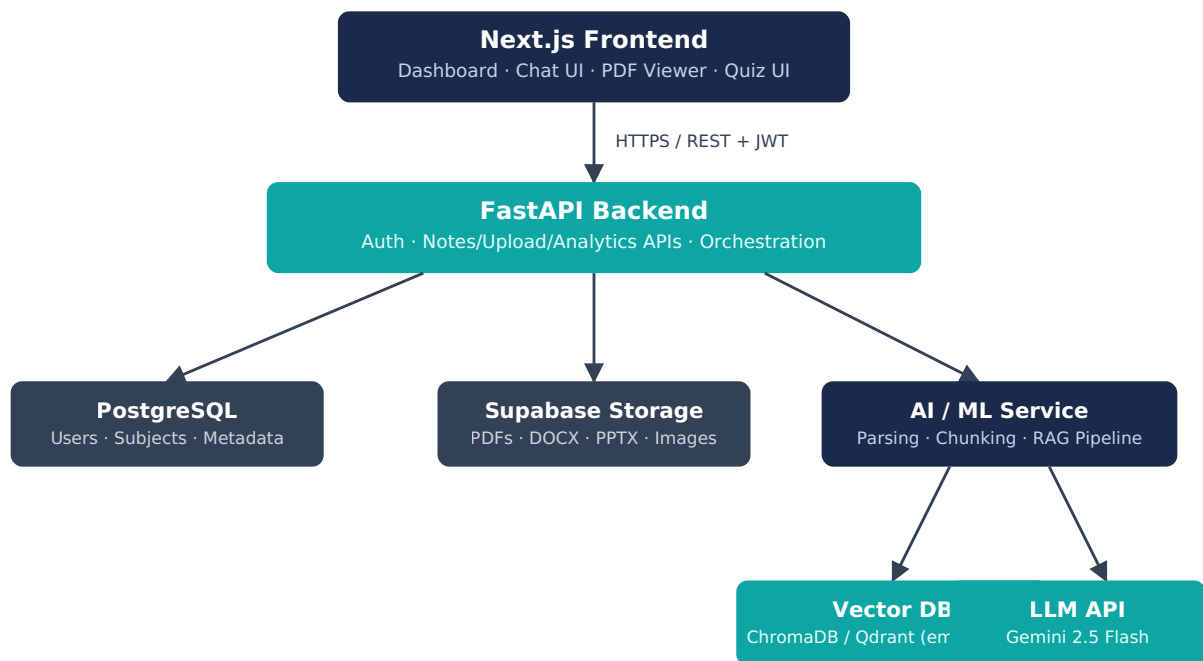


Fig 2.1 – High-level system architecture

2.3 What Each Layer Does

Layer	Responsibility	Tech
Frontend	Dashboard, notes manager, chat UI, PDF viewer, quiz UI, settings	Next.js, React, Tailwind, shadcn/ui
Backend API	Auth, file/notes/analytics APIs, request orchestration between frontend and AI service	FastAPI, SQLAlchemy, JWT, Google OAuth

AI/ML Service	Parsing, OCR, chunking, embeddings, RAG pipeline, quiz/flashcard/summary/planner generation	PyMuPDF, Sentence-Transformers, ChromaDB, Gemini API
Data Layer	Relational data + file storage + vector store	PostgreSQL, Supabase Storage, ChromaDB/Qdrant

2.4 How It Works End-to-End

Upload Flow

Client uploads a file → Backend stores the file in Supabase Storage and writes a metadata row in PostgreSQL → Backend hands the file to the AI Service → AI Service parses the document (OCR if it's a scanned image), splits it into chunks, generates embeddings, and stores the vectors in the Vector DB tagged with the user and document ID.

Chat Flow

User asks a question in the chat UI → Backend forwards the query with user/subject context to the AI Service → the query is embedded and the top-k relevant chunks are retrieved from the Vector DB → chunks are re-ranked and assembled into a context window → the LLM generates an answer using only that context → the answer is returned with citations (source document + page) → Backend logs the exchange to chat history → Frontend renders the answer with clickable citations that jump to the exact page in the PDF viewer.

03 / Development Phases

The build is broken into five phases across roughly 12 weeks. Each phase has a clear owner per workstream so no one blocks another for long.

Phase 1 – Foundation & Setup

Week 1–2

Repo scaffold and CI skeleton, authentication (JWT + Google OAuth), database schema v1 (users, subjects, notes metadata), base frontend shell with routing/layout/design tokens, and finalizing the AI stack via a small RAG spike on one sample document.

Sumit – auth, DB schema, repo/CI setup

Gaurav – frontend shell, layout, routing

Saurabh – finalize embeddings/vector DB choice, RAG spike

Phase 2 – Ingestion & Retrieval Core

Week 3–4

File upload API and storage integration, document parsing (PDF/DOCX/PPTX) with OCR for images, chunking + embedding pipeline into the vector DB, and the Notes Manager UI (create/edit/delete/tag/search).

Sumit – upload API, storage integration

Saurabh – parsing, OCR, chunking, embeddings

Gaurav – Notes Manager UI, Dashboard shell

Phase 3 – RAG Chat Experience

Week 5–7

Full RAG pipeline (retrieve → re-rank → context → generate → cite), chat API with persisted history, the chat UI (markdown, streaming, inline citations), and a PDF viewer that highlights and jumps to the cited page.

Saurabh – RAG pipeline, citation engine

Sumit – chat API, integration glue

Gaurav – chat UI + PDF viewer (paired with Sumit/Saurabh)

Phase 4 – AI Feature Expansion

Week 8–9

Summarizer, quiz generator with a scoring UI, flashcard generator, the study planner (subjects × deadlines × available hours), and analytics APIs feeding a progress dashboard.

Saurabh – summarizer, quiz/flashcard generators, planner logic

Sumit – analytics APIs, planner backend

Gaurav – Quiz UI, planner UI, settings screen

Phase 5 – Hardening, QA & Deployment

Week 10–12

Security pass (input validation, rate limiting, prompt-injection guardrails on the RAG pipeline, file-upload validation), testing and bug fixes, CI/CD to staging and production, and final documentation/demo prep.

Sumit – CI/CD, deployment

Saurabh – AI evaluation, safety guardrails

Gaurav – UI polish, responsiveness, bug bash

04 / Team Responsibilities

Work is split so the two heaviest, most interdependent pieces – auth/infra and the RAG pipeline – sit with the two more experienced members. Gaurav owns a real, visible slice of the product (dashboard, notes, settings) without being on the critical path, and pairs in on the harder chat/PDF-viewer screens once his core screens are stable.

Member	Primary Role	Owns	Tech Stack
Sumit	Backend & Infra / Integration Lead	Auth (JWT, Google OAuth, email verify, password reset, roles), DB schema, upload/notes/analytics APIs, wiring frontend ↔ AI service, CI/CD	FastAPI, PostgreSQL, SQLAlchemy, Alembic, Supabase Storage
Saurabh	AI / ML Lead	Document parsing, OCR, chunking, embeddings, vector DB, full RAG pipeline, summarizer, quiz/flashcard generators, study planner logic	PyMuPDF, pdfplumber, Sentence-Transformers, ChromaDB/Qdrant, Gemini API
Gaurav	Frontend (scoped)	Landing page, Dashboard, Notes Manager, Settings – plus paired work on Chat UI, PDF Viewer, and Quiz UI once comfortable	Next.js, React, Tailwind CSS, shadcn/ui

Note – Gaurav is newer to the stack, so his default workload is the CRUD-heavy, lower-complexity screens (forms, layouts, lists). The state-heavy screens (streaming chat, PDF highlighting, live scoring) are shared/paired rather than owned solo, so he isn't blocked and levels up through pairing rather than being handed the hardest UI cold.

05 / GitHub & Engineering Standards

5.1 Repository Structure

```
ai-study-assistant/
├── backend/          # Sumit – FastAPI app
│   ├── auth/
│   ├── api/
│   ├── database/
│   ├── models/
│   └── services/
├── frontend/        # Gaurav – Next.js app
│   ├── dashboard/
│   ├── chat/
│   ├── notes/
│   ├── quiz/
│   └── settings/
├── ai/              # Saurabh – RAG service
│   ├── parser/
│   ├── embeddings/
│   ├── rag/
│   ├── quiz/
│   └── planner/
├── docs/            # architecture, API spec, this document
├── .github/
│   ├── workflows/   # CI/CD pipelines
│   ├── ISSUE_TEMPLATE/
│   └── PULL_REQUEST_TEMPLATE.md
├── .env.example
├── CODEOWNERS
└── README.md
```

5.2 Branching Strategy

Branch	Purpose	Protection
main	Always production-deployable	Protected – PR + review + passing CI required, no force-push
develop	Integration branch for the current sprint	Protected – CI must pass
feature/<name>	One feature per branch, e.g. feature/rag-pipeline	None – merges into develop via PR
fix/<name>	Bug fixes	None – merges into develop via PR
release/<version>	Cut before a milestone/demo	Merges to main + develop

5.3 Commit Convention

Conventional Commits, so history and changelogs stay readable:

```
feat(ai): add citation engine to RAG pipeline
fix(auth): correct JWT refresh token expiry
docs: update API spec for /notes endpoint
chore: bump fastapi to 0.115
refactor(frontend): extract chat bubble component
```

5.4 Pull Request Process

- Every change goes through a PR – no direct pushes to `main` or `develop`.
- PR template requires: what changed, why, how it was tested, linked issue.
- Minimum 1 approving review from a teammate (not the author) before merge.
- CI (lint + tests) must pass before the merge button is enabled.
- Squash-merge into develop to keep history clean.

5.5 Issue Tracking & Project Board

Bug and feature issue templates, labeled by area (`backend` `frontend` `ai`) and priority. A GitHub Projects board tracks status: `Backlog` → `In Progress` → `In Review` → `Done` , mirroring the five phases above as milestones.

5.6 Code Ownership

```
/backend/ @sumit
/frontend/ @gaurav
/ai/ @saurabh
/docs/ @sumit @saurabh @gaurav
```

CODEOWNERS auto-requests the right reviewer when a PR touches a given folder.

5.7 CI/CD (GitHub Actions)

- **On every PR:** lint (ruff/eslint) + unit tests for the changed service.
- **On merge to develop:** build all three services, run integration tests, deploy to staging.
- **On merge to main:** deploy – Frontend to Vercel, Backend to Render, DB/Storage on Supabase.

5.8 Secrets & Environment

Real secrets (DB URL, JWT secret, Gemini API key, OAuth credentials) live only in GitHub Actions Secrets and each platform's environment settings – never committed. `.env.example` is committed with placeholder keys so any teammate can set up locally in minutes; `.env` itself is gitignored.

5.9 Documentation Standard

README covers: what the project is, local setup steps, how to run each of the three services, and links to this architecture document and the API spec. Each service folder gets its own short README for service-specific setup.